Halo 系统 API 参考文档

本文档详细列出了 Halo 系统提供的所有数据结构、调度器、优化器和 Worker 的详细信息。

目录

1. 数据结构
2. 调度器（Schedulers）
3. 优化器（Optimizers）
4. Worker 实现

---

1. 数据结构

1.1 核心类

类名	解释	重要参数/属性
Operator	工作流图中的操作节点，表示一个模型推理任务	`id`：唯一标识符 `input_ops`：前驱节点列表 `output_nodes`：后继节点列表 `prompt`：提示词字符串 `model_config`：ModelConfig 对象 `max_distance`：到终点的最长路径长度 `keep_cache`：是否保留 KV cache `data_parallel`：是否数据并行 `is_duplicate`：是否为复制节点 `duplicate_info`：复制信息 [rep_idx, rep_total]
Query	用户查询请求，跟踪查询在整个工作流中的状态和结果	`id`：查询唯一标识符 `prompt`：用户输入的提示词 `prompt_len`：提示词长度 `status`：查询状态（pending/running/finished） `priority`：优先级 `op_output`：字典，存储每个 OP 的输出（核心字段） `step`：当前执行步骤 `cache`：KV cache（TransformersWorker 使用） `create_time`：创建时间戳 `benchmark`：性能基准字典
ModelConfig	模型配置类，封装模型推理所需的所有参数	`model_name`：模型标识符（HuggingFace ID 或本地路径） `system_prompt`：系统级提示词 `temperature`：采样温度（0.0-1.0） `top_p`：Top-p 采样参数 `max_tokens`：最大生成 token 数 `max_batch_size`：最大批处理大小 `dtype`：数据类型（bfloat16/float16/float32） `use_chat_template`：是否使用聊天模板 `quantization`：量化配置（AWQ/GPTQ） `lora_config`：LoRA 适配器配置 `max_model_len`：最大序列长度 `min_tokens`：最小生成 token 数
ExecuteInfo	执行信息封装类，用于在优化器和 Worker 之间传递任务	`op`：Operator 对象 `query_ids`：查询 ID 列表 `prompts`：提示词列表（已包含父 OP 的输出）

类名	解释	重要参数/属性
Benchmark	性能基准类，记录操作执行的时间统计	`init_time`: 初始化时间（秒） `prefill_time`: Prefill 阶段时间（秒） `generate_time`: Generate 阶段时间（秒） `total_time()`: 计算总执行时间 `update(dict)`: 更新性能指标

---

2. 调度器（Schedulers）

2.1 调度器总览

调度器名称	方法名	方法解释	详细思想	性质
轮询调度	`schedule_rr`	按照拓扑顺序，将每个 OP 轮询分配到设备上	1. 对所有 OP 进行拓扑排序 2. 按顺序遍历每个 OP 3. 将 OP 分配到设备 d，处理所有查询 4. 设备索引循环递增：d = (d + 1) % device_cnt	**优点**：简单、快速 **缺点**：不考虑模型切换成本、缓存局部性 **复杂度**：O(V + E) **适用**：基准测试、简单工作流
层级调度	`schedule_by_levels`	将 OP 按依赖层级分组，同一层级的 OP 可以并行执行	1. 计算每个 OP 的层级（起始节点层级为 0） 2. 按层级分组，同一层级的 OP 收集到一起 3. 层级内使用轮询分配 4. 保证依赖关系：低层级总是先于高层级执行	**优点**：支持层级内并行、保证依赖关系 **缺点**：不考虑模型切换成本、可能跨设备传输数据 **复杂度**：O(V + E) **适用**：需要利用并行性的工作流
父节点共置	`schedule_colocate_parents`	优先将 OP 分配到其父节点所在的设备	1. 按拓扑顺序遍历每个 OP 2. 检查该 OP 的所有父节点 3. 如果某个父节点已分配到设备，则将该 OP 分配到同一设备 4. 如果没有父节点或父节点都未分配，使用轮询选择设备	**优点**：提高缓存局部性、减少跨设备传输 **缺点**：可能导致负载不均衡 **复杂度**：O(V + E) **适用**：需要利用 KV cache 的工作流
数据并行	`schedule_dp`	将查询 ID 固定分片到各设备，每个 OP 在所有设备上并行执行	1. 将查询 ID 列表均匀分成 device_cnt 个分片 2. 每个设备对应一个分片 3. 对每个 OP，在所有设备上都创建一个执行任务 4. 每个设备只处理分配给它的查询分片	**优点**：真正的并行执行、负载均衡 **缺点**：每个 OP 都需要在所有设备上加载模型、显存占用高 **复杂度**：O(V + E) **适用**：大批量查询、查询数量远大于设备数量
启发式调度（vLLM）	`schedule_heuristic`	综合考虑依赖关系、优先级、缓存局部性、数据并行等因素	1. 增长前沿：从起始节点开始，逐步扩展可执行的 OP 2. 依赖检查：只调度依赖已满足的 OP 3. 优先级选择：如果 OP 数量 > 设备数，按 max_distance 降序选择 4. 数据并行：如果 OP 数量 < 设备数，尝试恢复暂停的 OP 或复制 OP 5. 设备分配：优先将 OP 分配到父节点所在的设备	**优点**：综合考虑多种因素、比 Beam-Search 快、比简单调度智能 **缺点**：不考虑模型切换成本、贪心策略可能不是全局最优 **复杂度**：O(V + E + V*D)，D 为设备数 **适用**：需要综合考虑多种优化因素的工作流

调度器名称	方法名	方法解释	详细思想	性质
启发式调度（Transformers）	`schedule_workflows`	用于 Transformers Worker 的启发式调度，支持 KV cache 管理	1. 与 `schedule_heuristic` 类似的基础策略 2. 增加 cache 同步命令：dump_cache、get_cache、resume_cache 3. 管理跨设备 cache 传输 4. 处理 cache complete/merge 命令	**优点**：支持 KV cache 管理、跨设备 cache 传输 **缺点**：比 vLLM 版本复杂、需要额外的 cache 管理开销 **复杂度**：O(V + E + V*D) **适用**：使用 Transformers Worker 的工作流
Beam-Search 调度	`schedule_search`	基于 Beam-Search 的调度策略，维护多个候选状态，逐步扩展并选择最优执行计划	1. 维护一个 beam（多个部分调度状态） 2. 每一步：找到就绪的 OP（父 OP 已分配） 3. 选择最多	D

---

3. 优化器（Optimizers）

3.1 优化器对比

优化器名称	类名	优化办法	性质	适用场景
vLLM 优化器	`Optimizer_v` (halo_v.py)	1. **多进程架构**：为每个 GPU 创建独立的 Worker 进程 2. **依赖感知派发**：在派发任务前检查所有父 OP 输出是否就绪 3. **查询优化**：按优先级和提示词长度排序 4. **任务转换**：将父 OP 输出拼接到当前 prompt 5. **结果收集**：收集 Worker 结果并更新 Query.op_output 6. **性能监控**：统计延迟百分位数	**优点**： - 利用 vLLM 的高性能推理 - 支持连续批处理 - 自动管理 KV cache - 高吞吐量 **缺点**： - 不支持显式 cache 管理 - 不支持跨设备 cache 传输 **Worker**：vLLMWorker **调度器**：支持所有调度器	1. 高性能推理场景 2. 大批量查询处理 3. 不需要显式 cache 管理的场景 4. 单设备或多设备并行

优化器名称	类名	优化办法	性质	适用场景
Transformers 优化器	`Optimizer_t` (halo_t.py)	1. **多进程架构**：为每个 GPU 创建独立的 Worker 进程 2. **Cache 管理**：维护 cache 存储字典，支持跨设备 cache 传输 3. **依赖感知派发**：检查依赖并处理 cache 恢复 4. **任务转换**：支持 resume_cache 和 execute 两种命令 5. **Cache 同步**：处理 dump_cache 命令，存储 KV cache 6. **结果收集**：收集结果并更新 cache 和基准测试	**优点**： - 支持显式 KV cache 管理 - 支持跨设备 cache 传输 - 支持动态批处理调整 - 支持 CPU-GPU cache 卸载 **缺点**： - 性能可能不如 vLLM - 需要额外的 cache 管理开销 **Worker**：TransformersWorker **调度器**：使用 schedule_workflows	1. 需要显式 cache 管理的场景 2. 需要跨设备 cache 传输的场景 3. 需要动态批处理调整的场景 4. 需要 CPU-GPU cache 卸载的场景

3.2 优化器主要方法

方法名	所属优化器	功能说明	重要参数
`__init__`	两者	初始化优化器，加载配置并创建 Worker 进程	`config_path`: YAML 配置文件路径
`schedule`	Optimizer_v	根据选定的调度策略生成执行计划	`queries`: 查询列表 `strategy`: 调度策略（search/heuristic/rr/levels/colocate_parents/dp）
`schedule`	Optimizer_t	通过 schedule_workflows 构建每个设备的工作流	`queries`: 查询列表
`execute`	两者	按照工作流将任务推送给 Worker，收集结果	`queries`: 查询列表（可选） `return_queries`: 是否返回查询列表 `skip_exit`: 是否跳过退出命令
`_optimize_queries`	两者	对查询进行排序优化（按优先级和长度）	`queries`: 查询列表
`_create_workers`	两者	为每个可见的物理 GPU 创建 Worker 进程	无参数
`print_latency_percentiles`	Optimizer_v	计算并打印所有查询的延迟百分位数（P50/P95）	无参数
`exit`	两者	关闭所有队列并等待进程结束	无参数
`_cache_resume`	Optimizer_t	返回指定 OP 和查询 ID 子集的缓存 KV 片段	`op_id`: Operator ID `query_ids`: 查询 ID 列表

---

4. Worker 实现

4.1 Worker 对比

Worker 名称	类名	解释	特点	适用场景
vLLM Worker	`vLLMWorker` (worker_v.py)	基于 vLLM 的 Worker 进程，绑定到特定 GPU，负责执行模型推理任务	1. **高性能推理**：利用 vLLM 的连续批处理和 PagedAttention 2. **自动 cache 管理**：vLLM 自动管理 KV cache，无需显式操作 3. **模型切换**：按需加载和切换模型 4. **聊天模板支持**：可选使用 HuggingFace 聊天模板 5. **批量生成**：通过 vLLM.generate 执行批量推理 6. **简单接口**：只支持 execute 和 exit 命令	**优点**: - 高吞吐量 - 自动优化批处理 - 高效的 KV cache 管理 - 简单易用 **缺点**: - 不支持显式 cache 管理 - 不支持跨设备 cache 传输 - 不支持动态批处理调整

Worker 名称	类名	解释	特点	适用场景
Transformers Worker	`TransformersWorker` (worker_t.py)	基于 Transformers 库的 Worker，支持显式的 KV cache 管理和跨设备传输	1. **显式 cache 管理**：支持 dump_cache、get_cache、resume_cache 命令 2. **跨设备 cache 传输**：通过 communication_queues 传输 cache 3. **CPU-GPU cache 卸载**：后台线程在 CPU 和 GPU 之间卸载/预取 cache 4. **动态批处理调整**：根据 GPU 利用率动态调整批处理大小 5. **多模型支持**：支持单模型或多模型模式 6. **流式生成**：使用批量解码和 cache 复用进行流式生成	**优点**: - 灵活的 cache 管理 - 支持跨设备 cache 传输 - 支持动态批处理调整 - 支持 CPU-GPU cache 卸载 **缺点**: - 性能可能不如 vLLM - 需要额外的 cache 管理开销 - 实现更复杂

4.2 Worker 主要方法

方法名	所属 Worker	功能说明	重要参数
`__init__`	两者	初始化 Worker，绑定到指定 GPU	`id`: Worker ID `physical_gpu_id`: 物理 GPU ID `cmd_queue`: 命令队列 `result_queue`: 结果队列 `communication_queues`: 通信队列（仅 TransformersWorker） `models`: 模型列表（仅 TransformersWorker）
`init_op`	两者	为每个 OP 初始化运行时状态，切换模型/tokenizer（如需要）	`op`: Operator 对象
`execute`	两者	执行一个执行任务，返回结果	`exe_info`: ExecuteInfo 对象
`run`	两者	Worker 的主循环，持续监听命令队列并执行任务	`debug`: 是否在调试模式下运行
`exit`	两者	清理资源并退出 Worker	无参数
`dump_cache`	TransformersWorker	转储 KV cache 到 CPU	无参数
`get_cache`	TransformersWorker	从其他设备获取 KV cache	`source_device`: 源设备 ID `op_id`: Operator ID `query_ids`: 查询 ID 列表
`resume_cache`	TransformersWorker	恢复 KV cache 并继续执行	`cache`: KV cache 字典

---

5. 使用示例

5.1 基本使用流程

```
from halo.optimizers import Optimizer_v
from halo.components import Query

# 1. 创建优化器
opt = Optimizer_v("templates/adv_reason_3.yaml")

# 2. 创建查询
queries = [Query(i, "What is Machine Learning?") for i in range(10)]

# 3. 调度（选择策略）
opt.schedule(queries, strategy="heuristic")
```

```
# 4. 执行
queries = opt.execute(return_queries=True)

# 5. 查看结果
for q in queries:
    print(f"Query {q.id} result: {q.op_output}")

# 6. 清理
opt.exit()
```

5.2 调度策略选择

场景	推荐调度器	理由
基准测试	`rr`	简单、快速
层级结构清晰	`levels`	支持层级内并行
需要 cache 优化	`colocate_parents`	提高缓存局部性
大批量查询	`dp`	真正的并行执行
综合考虑多种因素	`heuristic`	平衡性能和复杂度
需要最优解	`search`	近似最优，但计算开销大

---

6. 性能指标

6.1 Benchmark 字段说明

- **init_time**: 初始化时间，包括模型加载、参数设置等
- **prefill_time**: Prefill 阶段时间，处理输入 prompt 的时间
- **generate_time**: Generate 阶段时间，生成新 token 的时间
- **total_time**: 总执行时间 = init_time + prefill_time + generate_time

6.2 延迟百分位数

- **P50（中位数）**: 50% 的查询延迟低于此值
- **P95（95 百分位）**: 95% 的查询延迟低于此值

---

7. 注意事项

1. **依赖关系**：系统会自动检查依赖关系，确保父 OP 输出就绪后才执行子 OP
2. **模型切换**：切换模型会有性能开销，调度器会尽量将相同模型的 OP 分配到同一设备
3. **Cache 管理**：vLLM Worker 自动管理 cache，Transformers Worker 需要显式管理
4. **批处理**：系统会自动进行批处理，提高 GPU 利用率
5. **多进程**：每个 GPU 对应一个独立的 Worker 进程，通过队列通信

文档版本: 1.0
最后更新: 2025-01-15